TACTIVE INTERNSHIP ASSESSMENT

SafeSeat

Reserved-Quota Safety Seating for Bus Travel

A configurable reserved-pair safety mechanism for bus booking, built with an AI-orchestrated design → test → change loop.

Bala Hari Haran P

B.Tech CSE, SRM Institute of Science and Technology, Trichy

github.com/BalaHariHaran2505/bus-reservation-system

Problem & design goals

THE PROBLEM

Bus reservation systems need a real safety option for female passengers — without a rule that tries to infer who “belongs” together and ends up blocking ordinary travellers.

RISKY APPROACH

Pure adjacency detection

- Tries to guess relationships between strangers
- Blocks legitimate couples & families booking separately

DESIGN RESPONSE

Reserve configurable seat pairs, keep the rest fully open

OUTCOME

- **18** deterministic allocator tests
- **4** safety states: general, reserved, booked-F, booked-M
- **✓** receipts, cancellation, admin controls, full-bus messaging

How the safety rule works

Solo female

Reserved seat first

Male / group

General seating only, no restriction

Manual pick: general

Anyone can book it

Manual pick: reserved

Female passengers only

DECK-INDEPENDENT CONFIGURATION

lower_reserved_pairs and upper_reserved_pairs are set independently per deck — avoiding relationship-guessing while still giving a concrete, predictable safety option.

SEAT COLOUR KEY

- General — free
- Ladies reserved — free
- Booked — female
- Booked — male

Architecture

Browser
Jinja HTML +
minimal JS

→

Flask app.py
Routes, validation,
auth, DB writes

→

Seat allocator
Pure Python
business rules

→

SQLAlchemy
Bus / SeatRow /
BookingLog

→

SQLite / MySQL
DATABASE_URL
supported

- seat_allocator.py is framework-free and independently unit-testable.
- app.py converts persisted SeatRow state into allocator Seat objects and commits successful bookings.
- CSRF protection, server-side validation, environment-based secrets, and password-gated admin routes are included throughout.

DESIGN

Booking flow & rule matrix

1. **Enter details** — Name, phone, gender, optional group size
2. **Choose mode** — Auto-assign or select a specific free seat
3. **Apply rules** — Reserved quota enforced by the allocator
4. **Persist** — Booking + audit log written via SQLAlchemy
5. **Receipt** — Seat, fare and total shown

RULE MATRIX

Solo female	Male	Group	Manual + general	Manual + reserved
Reserved first	General only	General only	Anyone	Female only

Testing & evidence

18

tests collected

18

passed / 0 failed

0.07s

final local run

RED RUN

- Deliberately incorrect reserved-seat assertion
- Male attempts a Ladies-reserved seat
- Real ReservedSeatError failure captured

CI

- GitHub Actions workflow present
- Tests execute automatically on every push
- Latest workflow run shown as successful

AI change loop: failure → correction → redesign

1

Attempt 1

FAILED

strict_safety added to auto_assign only. manual_assign untouched → TypeError.

2

Attempt 2

17 / 17

Extended to manual booking + night-bus handling. Full suite green.

3

Manual test

DISCOVERY

Found the adjacency rule blocked legitimate couples & families.

4

Attempt 3

18 / 18

Replaced adjacency detection with reserved-quota pairs; fixed global allocation; added deck config.

KEY LESSON

Tests can validate a flawed design. Automated tests never caught the adjacency-rule bug — they only checked the rule did what it was built to do. Manual testing found it, and the fix was a redesign, not more tests.

Security & engineering choices

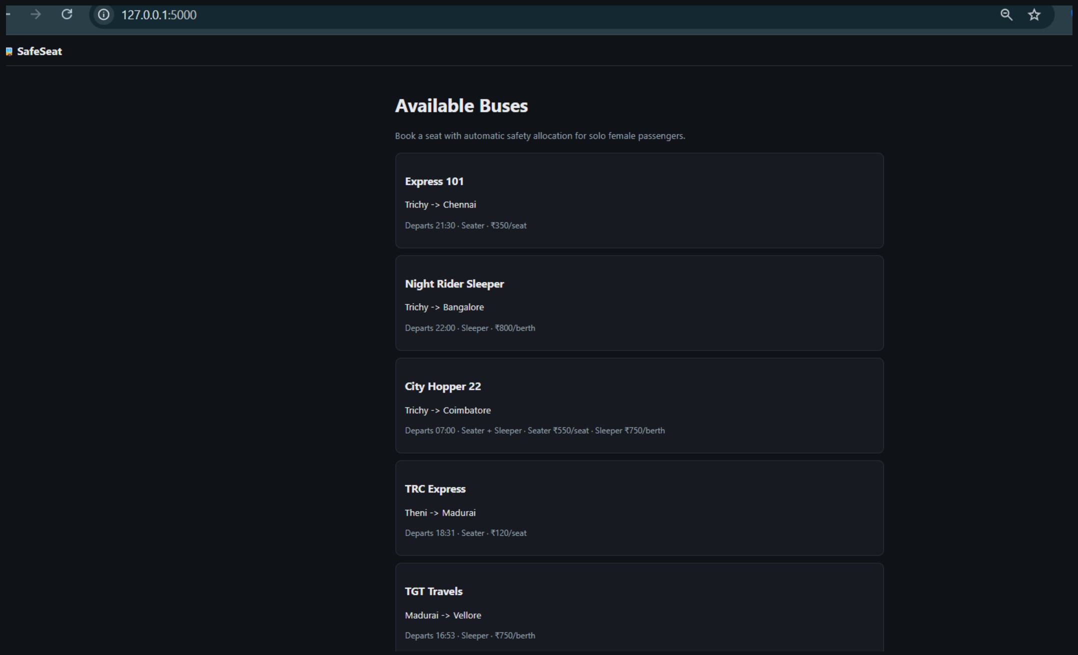
SECURITY

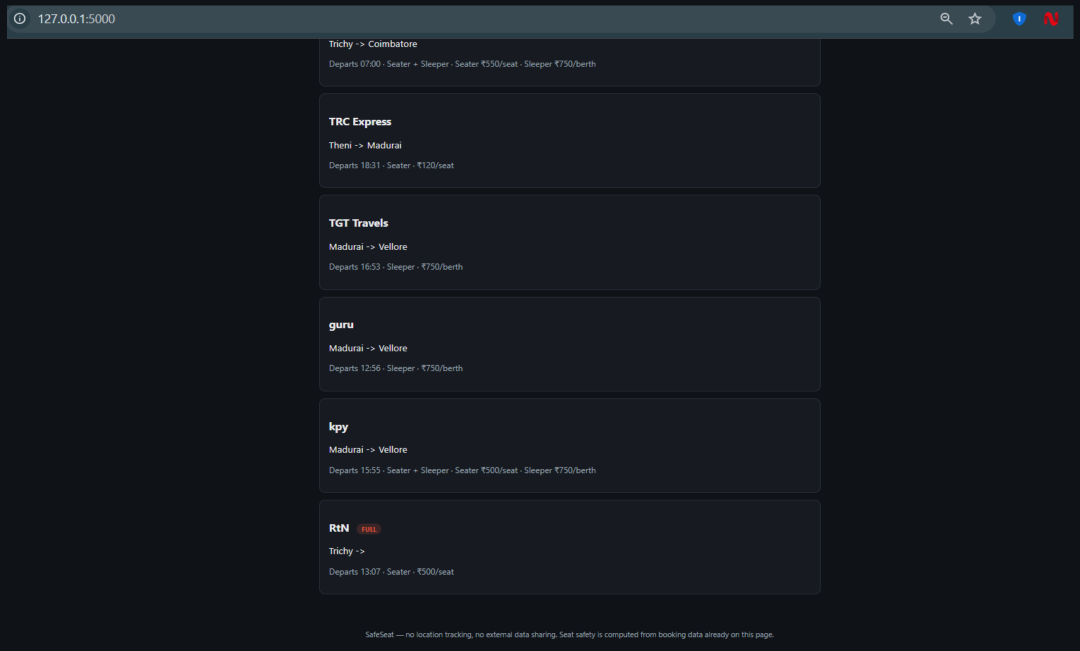
- ✓ CSRF protection on all state-changing forms
- ✓ Server-side booking validation
- ✓ SQLAlchemy ORM instead of raw SQL
- ✓ Secrets kept in .env; .env is git-ignored
- ✓ Admin dashboard is password-gated

ENGINEERING

- → SQLite for zero-setup local execution
- → MySQL supported through DATABASE_URL
- → Framework-free allocator for deterministic tests
- → Server-rendered HTML keeps the frontend simple
- → Audit log records every booking/cancellation

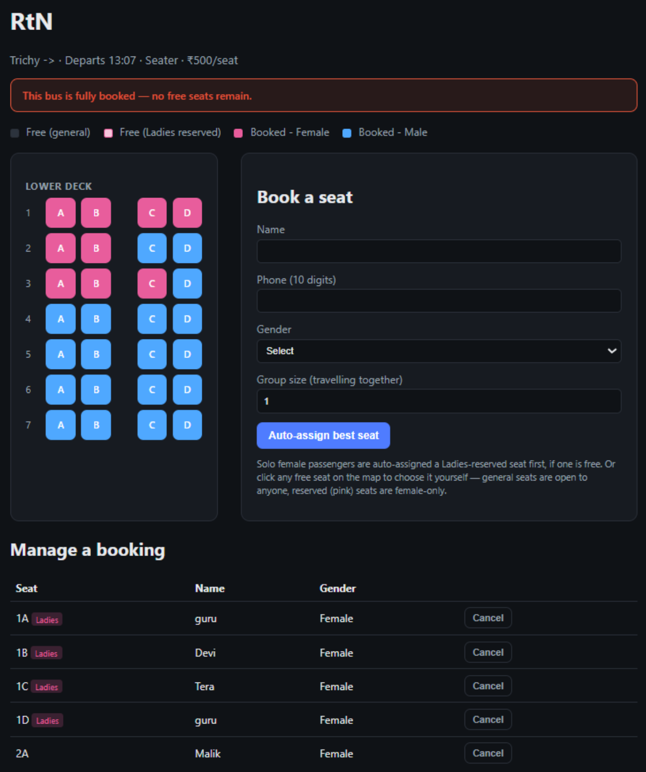
Bus listing, sold-out state

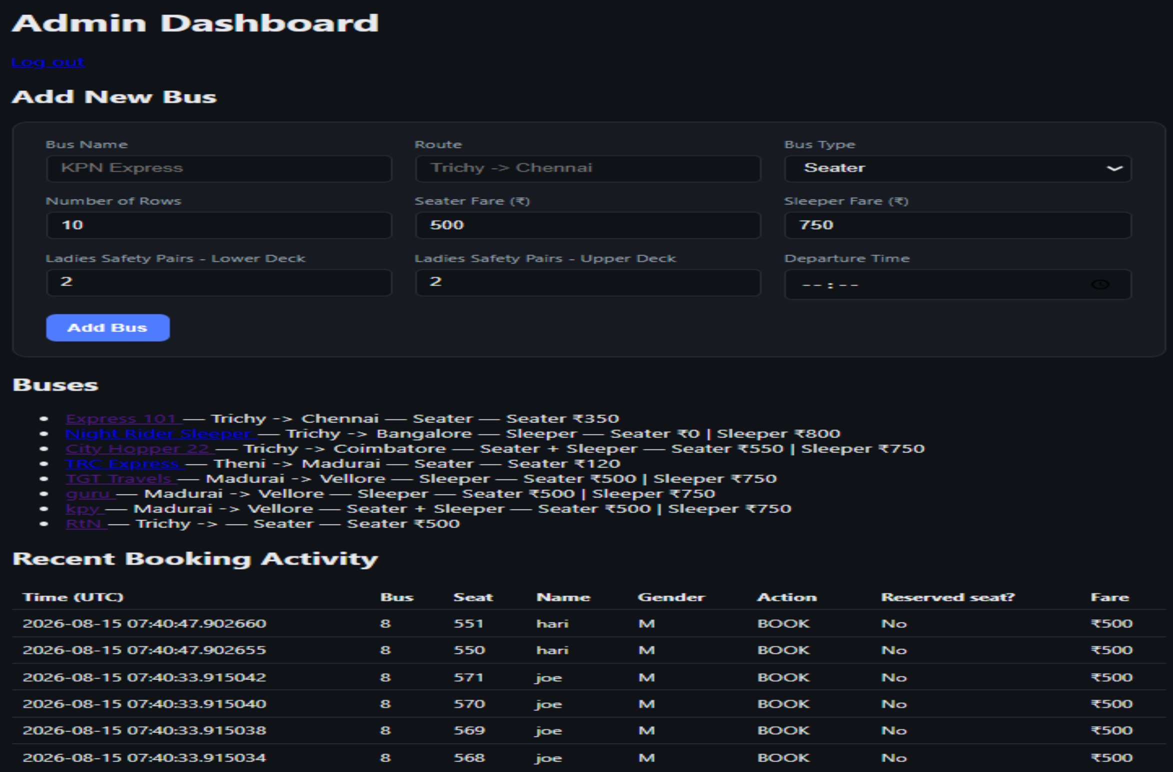




The bus listing now visibly marks a sold-out bus as FULL — no need to open the booking page to find out. Scrolling the list (right) shows the badge in place next to the bus name.

Seat map, full-bus state & admin





Reserved Ladies seats are visually distinct on the map, and the fully-booked bus shows the explicit banner rather than failing silently. The admin dashboard configures lower/upper deck reserved pairs per bus.

What changed, and what's next

WHAT CHANGED AFTER THE FINAL CODE ITERATION

- Bus availability UX: fully booked buses are now visibly marked FULL on the bus list.
- Booking UX: the bus page explicitly reports when no free seats remain.
- Core behavior remains stable — the latest pytest run is 18/18 passed; these are UI refinements on top of the already-tested allocator, not a new change-loop attempt.

KNOWN LIMITATIONS

- No real-time seat locking for concurrent bookings
- No payment gateway
- Browser / E2E tests are not yet included

NEXT ITERATION

- DB row locking / reservation holds
- Selenium or Playwright UI tests
- Admin editing of reserved-pair counts
- Stronger auth if productionized

AI TOOL USED

Claude (Anthropic)

Used for application code, allocator logic, pytest authoring, change-loop iterations, technical documentation, and this presentation.

SafeSeat

18/18 tests passed

GitHub Actions CI

AI change-loop evidence

Live UI demo

github.com/BalaHariHaran2505/bus-reservation-system

Bala Hari Haran P | B.Tech CSE, SRM Institute of Science and Technology